

Scaling Laws

Why bigger models are a safe bet · video 2:25

This lesson answers one question: why do labs keep training bigger models on more data? Because the loss falls predictably. Six tiny GPTs on the same data line up on a straight line in log-log axes: a **power law**, $\text{loss} \approx 4.24 \times N^{-0.071}$. One model trained on doubling amounts of data gives another, $\text{loss} \approx 10.68 \times \text{tokens}^{-0.121}$. The biggest model falls slightly above its line, short of data. Fitted on small runs, such laws let labs predict the loss of much larger ones.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 5).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. Loss vs model size	▶ 0:08-1:10	2	☐
2. Loss vs data	▶ 1:10-1:32	2	☐
3. Real scaling laws	▶ 1:32-2:17	3	☐

Before you start: Deep Dive episode 17 (the loss) and Foundations (logarithms). Code: `code/d23_scaling_laws` (PyTorch only; about 45 minutes on a CPU).

CONCEPT 1 OF 3

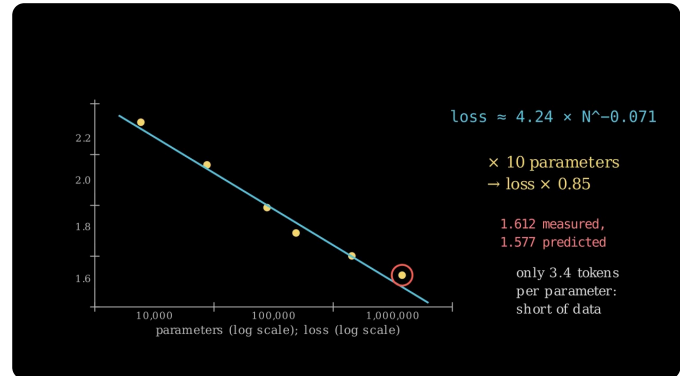
Loss vs model size

▶ Video 0:08–1:10

Six tiny GPTs are trained on the same 4,096,000 tokens, from 7,697 to 1,198,273 parameters (excluding embeddings). Validation loss: 2.280, 2.070, 1.879, 1.774, 1.684, 1.612. With logarithmic axes the points lie almost on a straight line: a **power law**, $\text{loss} \approx 4.24 \times N^{-0.071}$. Every 10× more parameters multiplies the loss by about **0.85** ($10^{-0.071}$).

six tiny GPTs, the same 4,096,000 training tokens

parameters (excluding embeddings)	val loss
7,697	2.280
27,617	2.070
88,097	1.879
154,305	1.774
453,857	1.684
1,198,273	1.612



▶ 0:37 Six tiny GPTs, 7,697 to 1,198,273 parameters, the same 4,096,000 tokens: loss 2.280 down to 1.612.

▶ 1:09 In log-log axes, a straight line: $\text{loss} \approx 4.24 \times N^{-0.071}$; the biggest model sits above it.

The biggest model sits a little above the line: **1.612** measured, 1.577 predicted. A likely reason: it saw only about 3.4 training tokens per parameter, so it is starting to run short of data.

KEY IDEA

A power law $y = a \cdot x^b$ is a straight line in log-log axes: $\log y = \log a + b \cdot \log x$.

Check 1: Loss vs model size

Answer all 2 questions before looking at the answers. Pass mark: all correct.

1.1 CALCULATION

What loss does the fitted law predict for 10 million parameters?

.....

1.2 CALCULATION

By what factor does the loss shrink for 100 times more parameters?

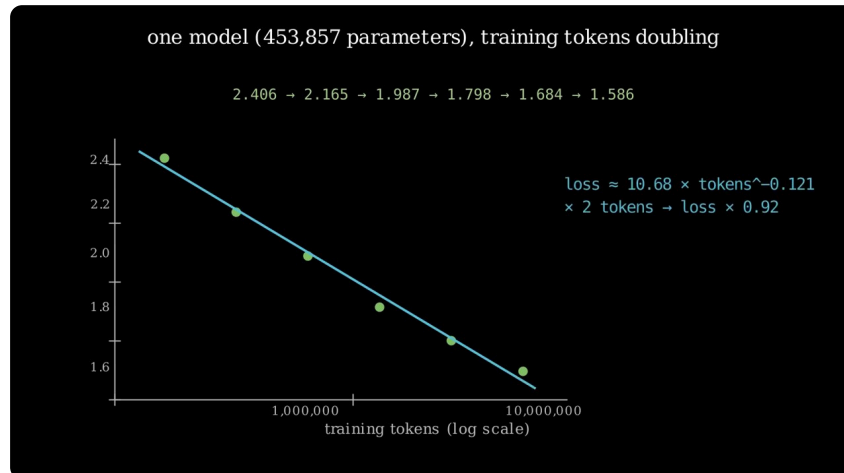
.....

CONCEPT 2 OF 3

Loss vs data

▶ Video 1:10-1:32

One model (width 96, 4 layers, 453,857 parameters) is measured as its training tokens double from 256,000 to 8,192,000: 2.406, 2.165, 1.987, 1.798, 1.684, 1.586. Another power law: $\text{loss} \approx 10.68 \times \text{tokens}^{-0.121}$; every doubling multiplies the loss by about **0.92**.



▶ 1:30 One 453,857-parameter model as its tokens double: 2.406 → 1.586; $\text{loss} \approx 10.68 \times \text{tokens}^{-0.121}$.

KEY IDEA

More data lowers the loss predictably too, until the model's size becomes the limit.

Check 2: Loss vs data

Answer all 2 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

By the data law, what is the loss at 16,384,000 tokens (one more doubling)?

.....

2.2 CALCULATION

Four times more tokens multiplies the loss by about how much?

.....

CONCEPT 3 OF 3

Real scaling laws

▶ Video 1:32-2:17

Research labs found the same shape on real language models, over many orders of magnitude of size, data and compute. A widely cited result, from the **Chinchilla** paper (2022), is that for a fixed compute budget the best results come from about **20 training tokens per parameter**. Our biggest model had 3.4.

That is what makes huge training runs possible to plan: fit the law on small, cheap runs and predict the loss of a model a thousand times larger before spending the money. The laws describe the loss, not every ability, and they bend when something runs short, as our biggest model showed. In code, fitting one is fitting a straight line through the logarithms.

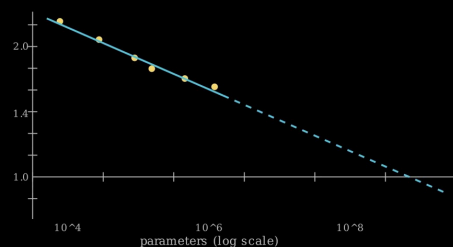
real language models: the same shape,
over many orders of magnitude of size, data and compute

Chinchilla (2022): for a fixed compute budget,
about 20 training tokens per parameter

our biggest model: $4,096,000 / 1,198,273 \approx 3.4$

▶ 1:47 The same shape on real language models; Chinchilla: about 20 tokens per parameter for a fixed compute budget.

fit on small, cheap runs → predict a 1,000× larger model



▶ 1:59 Fit on small, cheap runs; extrapolate to a model 1,000× larger.

KEY IDEA

Scaling laws turn “bigger is better” into a number you can plan with.

Check 3: Real scaling laws

Answer all 3 questions before looking at the answers. Pass mark: all correct.

3.1 CALCULATION

At about 20 tokens per parameter, how many tokens would our 1,198,273-parameter model need? And a 7-billion-parameter model?

3.2 TRUE OR FALSE

“A scaling law guarantees a bigger model will be better at every task.”

True ☐ False ☐

3.3 HANDS-ON CODE

Try it yourself. Fit the law to only the four smallest models. How close is its prediction for the 1,198,273-parameter model?

```
a, s = fit_power_law(*zip(*sizes[:4]))
print(a * 1198273 ** s)
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. Loss vs model size · rewatch ▶ 0:08-1:10

1.1 About 1.35. $4.24 \times (10^7)^{-0.071} \approx 1.350$ (if the data were sufficient).

1.2 About 0.72. $0.85^2 \approx 0.72$, or $100^{-0.071}$.

2. Loss vs data · rewatch ▶ 1:10-1:32

2.1 About 1.43 predicted. $10.68 \times 16,384,000^{-0.121} \approx 1.431$. Note that the measured 1.586 at 8.2M tokens already sits above the line's value.

2.2 About 0.85. $0.92^2 \approx 0.846 (= 4^{-0.121})$.

3. Real scaling laws · rewatch ▶ 1:32-2:17

3.1 About 24 million; 140 billion. $20 \times 1,198,273 \approx 23,965,460$; $20 \times 7 \times 10^9$.

3.2 False. It predicts the average loss; specific abilities may not follow it smoothly.

3.3 The four-point fit is $4.81 \times N^{-0.083}$, predicting 1.506: well below the measured 1.612, because the big model is short of data (computed from the measured losses). Extrapolations assume nothing else (here, data) becomes the limit.